

PRINCIPIOS DE SISTEMAS OPERATIVOS

Preámbulo: Funciones y componentes de los Sistemas Operativos



Objetivos de la unidad

- **Definir** objetivos, funciones y evolución histórica de los Sistemas Operativos
- **Identificar** los componentes fundamentales de un SO moderno
- **Analizar** problemas clásicos que resuelven los SO
- **Comparar** arquitecturas y tipos de SO actuales
- **Contextualizar** el rol del SO en sistemas contemporáneos (nube, móviles, edge)

Evolución Histórica: De los Mainframes a la Nube

Década	Tecnología	SO Representativos	Característica Clave
1950s	Mainframes	GM-NAA I/O, IBSYS	Procesamiento por lotes
1960s	Tiempo compartido	CTSS, MULTICS	Multiprogramación
1970s	Minicomputadoras	UNIX, CP/M	Portabilidad, kernel en C
1980s	Computadoras personales	MS-DOS, Mac OS	Interfaz de usuario
1990s	Cliente-servidor	Windows NT, Linux	Redes, 32/64 bits
2000s	Móviles/Web	Windows XP, Android, iOS	Conectividad, touch
2010s	Cloud/Contenedores	Windows 10, Docker, K8s	Virtualización ligera
2020s	Edge/IoT	Windows 11, ROS 2, RHEL	Distribuido, tiempo real

Referencia: Tanenbaum, A. y Bos, H. *Modern Operating Systems*, 5th ed., Pearson, 2023, Cap. 1

¿Qué es un Sistema Operativo?

Definición Multidimensional:

1. Administrador de recursos

- Gestiona CPU, memoria, almacenamiento, dispositivos
- Asigna recursos de manera eficiente y justa

2. Máquina virtual/extendida

- Abstrae la complejidad del hardware
- Proporciona interfaces uniformes a programas

3. Controlador/proveedor de servicios

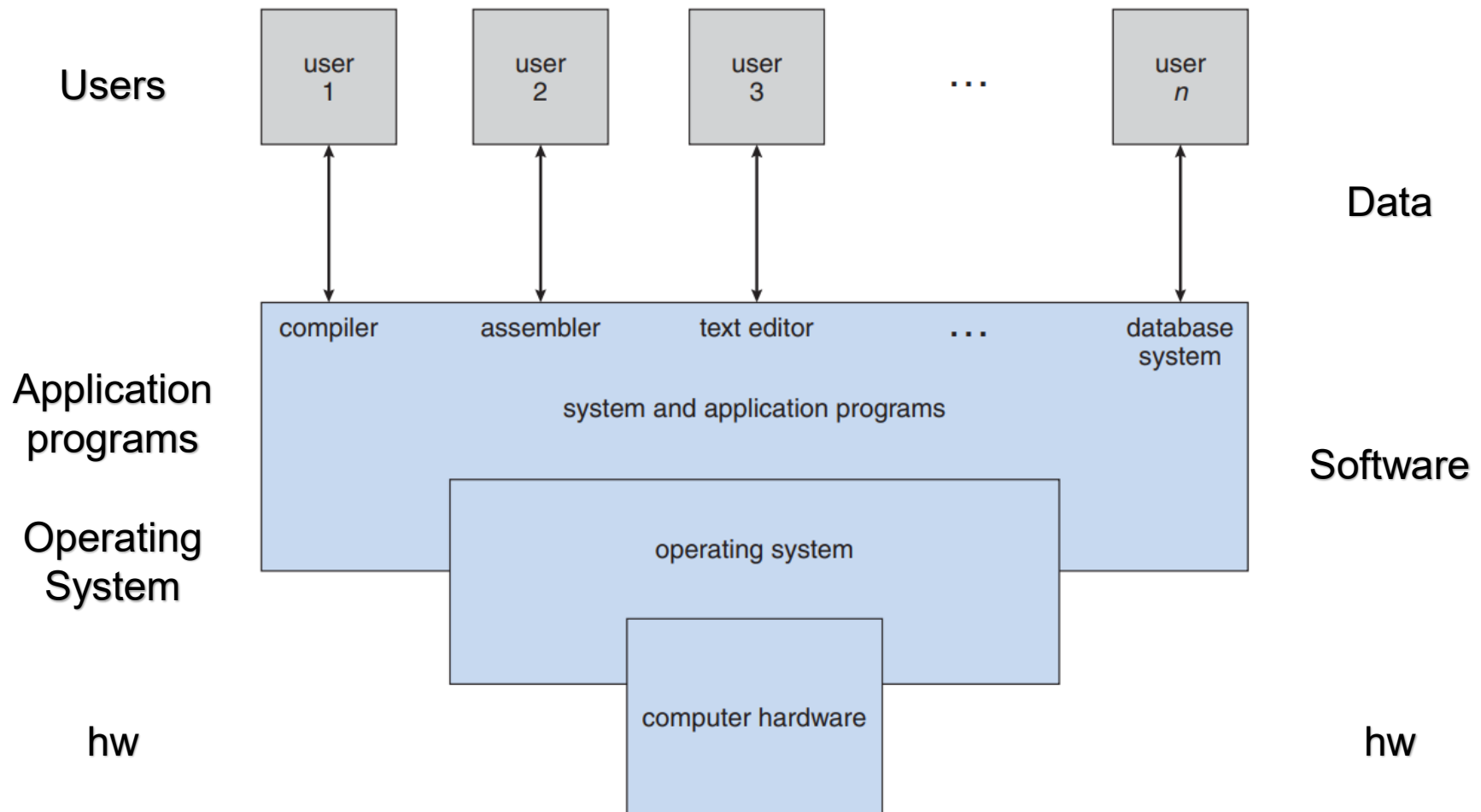
- Ofrece servicios comunes a aplicaciones
- Garantiza operación confiable y segura

"El SO es el software que hace usable el hardware"

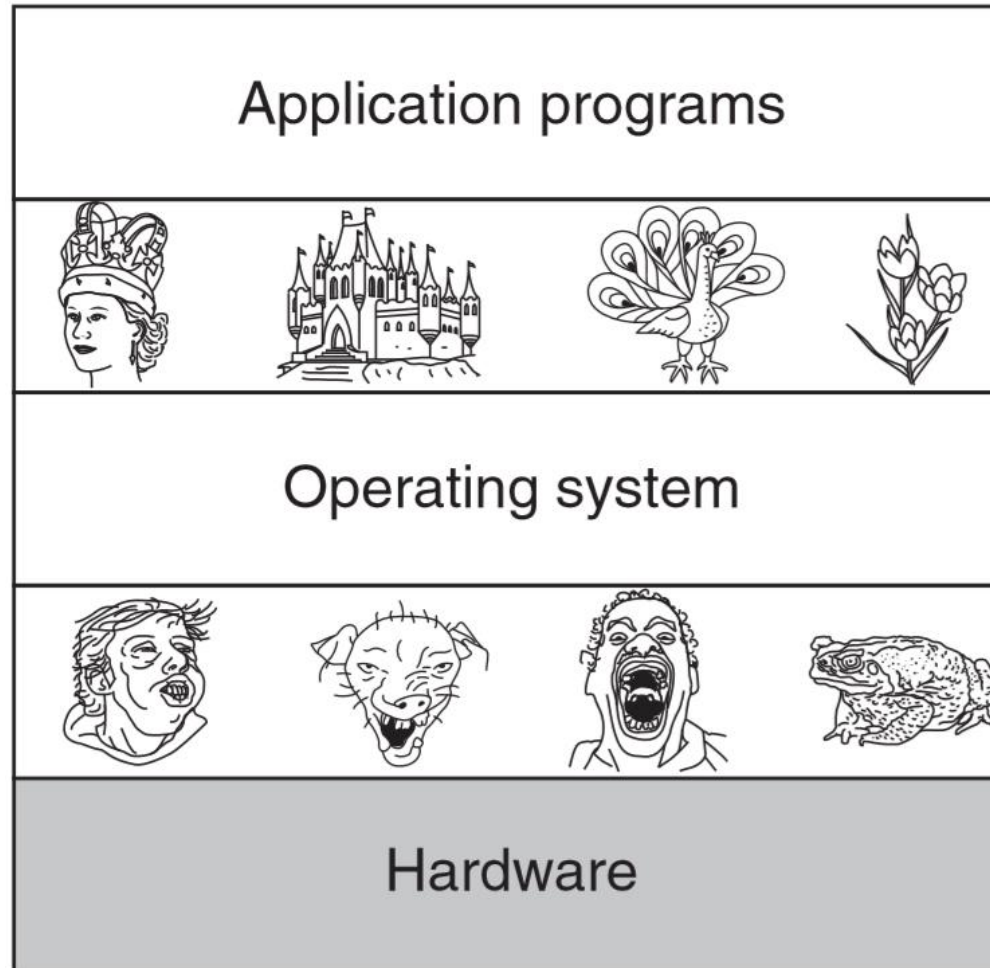
— Silberschatz, A., et al. *Operating System Concepts*, 11th ed., 2022

Función de los Sistemas Operativos

Visión abstracta de los componentes



Visión abstracta del sistema



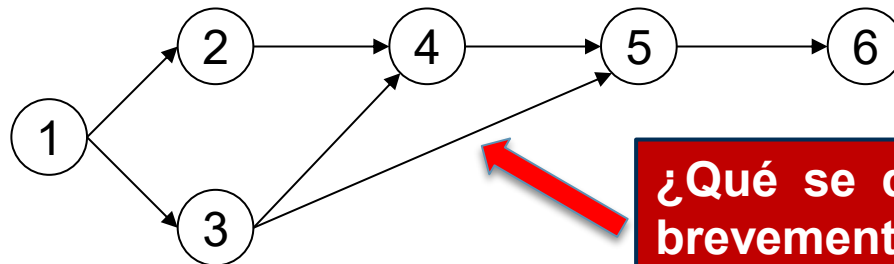
Tanenbaum, A. S., Bos, H. (2014). *Modern Operating Systems*. Boston, MA: Pearson. ISBN: 978-0-13-359162-0, p. 5.

Problemas/situaciones que gestiona un SO

Demanda de recursos, bloqueos, ejecución concurrente

Problemas que resuelve un S.O. (I)

- Un **programa en ejecución** es un conjunto de **procesos en ejecución**

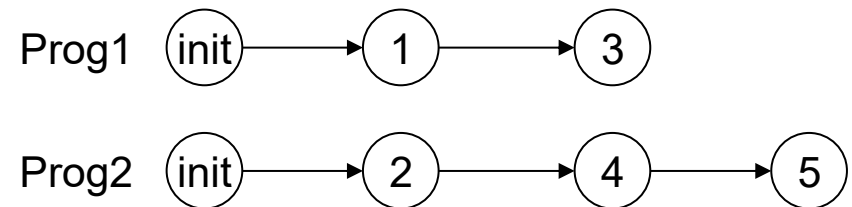
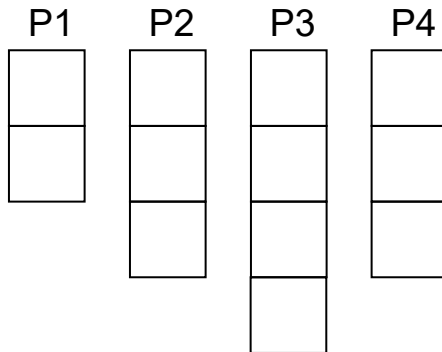
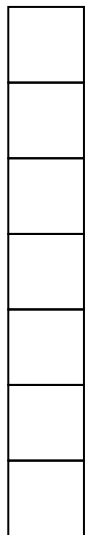


- Cómo sabe el proceso 2 que el proceso 1 ya terminó.
- Dado que 2 y 3 se pueden ejecutar en paralelo, ¿quién se ejecuta primero?
- Si 2 y 3 ya terminaron, ¿cómo se enteró 4 de este suceso?

Problemas que resuelve un S.O. (II)

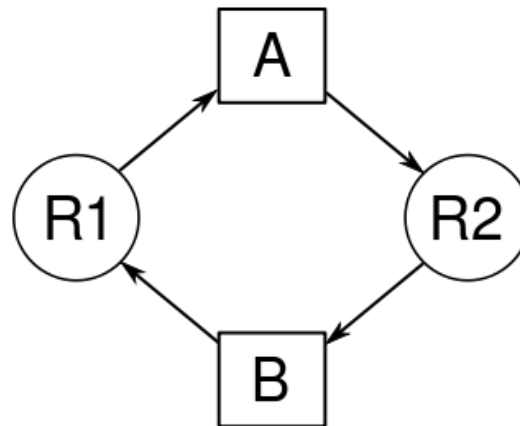
- Los procesos requieren de recursos.
- Si ambos programas se ejecutan en paralelo. ¿en qué orden se debe de asignar los recursos a los procesos?
- Considere que mientras un programa no termina, los procesos asociados no pueden liberar los recursos. Una propiedad deseable de un SO es la multiprogramación.

memoria



Problemas que resuelve un S.O. (III)

- El siguiente problema tiene varias denominaciones: ***Bloqueo mutuo, interbloqueo, traba mortal, deadlock, abrazo mortal.***
- Se presenta cuando el μP no puede asignar un recurso compartido a un conjunto de procesos pendientes de terminar.
- C/u de los procesos (o hilos de ejecución) tienen un recurso asignado, pero requieren del recurso asignado al otro proceso para continuar.





Componentes Principales de un SO Moderno

1. Gestor de Procesos

- **¿Qué es un proceso?**

Programa en ejecución + estado + recursos

Responsabilidades:

- Creación/destrucción (fork/exec/exit)
- Scheduling (planificación)
- Sincronización (semáforos, mutex)
- Comunicación (pipes, sockets, memoria compartida)

Ejemplo moderno: Contenedores (namespaces, cgroups)

2. Gestor de Memoria

- **Jerarquía gestionada:**

Registros → Cache L1/L2/L3 → RAM → SSD/NVMe → Cloud Storage

Responsabilidades:

- Asignación/liberación
- Paginación/segmentación
- Memoria virtual (swap)
- Caché y buffers

Ejemplo moderno: Memoria no volátil (Optane), DAX

3. Gestor de E/S

- **Arquitectura en capas:**

Aplicación → API de E/S → Drivers genéricos → Drivers específicos → Hardware

Responsabilidades:

- Buffering y caching
- Scheduling de discos
- Gestión de interrupciones
- Soporte plug-and-play

Ejemplo moderno: NVMe drivers, GPU compute

4. Sistema de Archivos

- **Abstracciones:**

- Archivo: Datos + metadatos
- Directorio: Organización jerárquica
- Punto de montaje: Integración de dispositivos

Responsabilidades:

- Organización en disco (FAT, NTFS, ext4, ZFS)
- Permisos y control de acceso
- Recuperación ante fallos (journaling)
- Caché de disco

Ejemplo moderno: Sistemas de archivos distribuidos
(HDFS, Ceph)

5. Subsistema de Red

- **Responsabilidades:**

- Protocolos de red (TCP/IP stack)
- Interfaces de red (NIC drivers)
- Firewall y filtrado
- API de sockets

Ejemplo moderno: Virtualización de red (OVS, eBPF)

6. Seguridad y Protección

- **Mecanismos:**

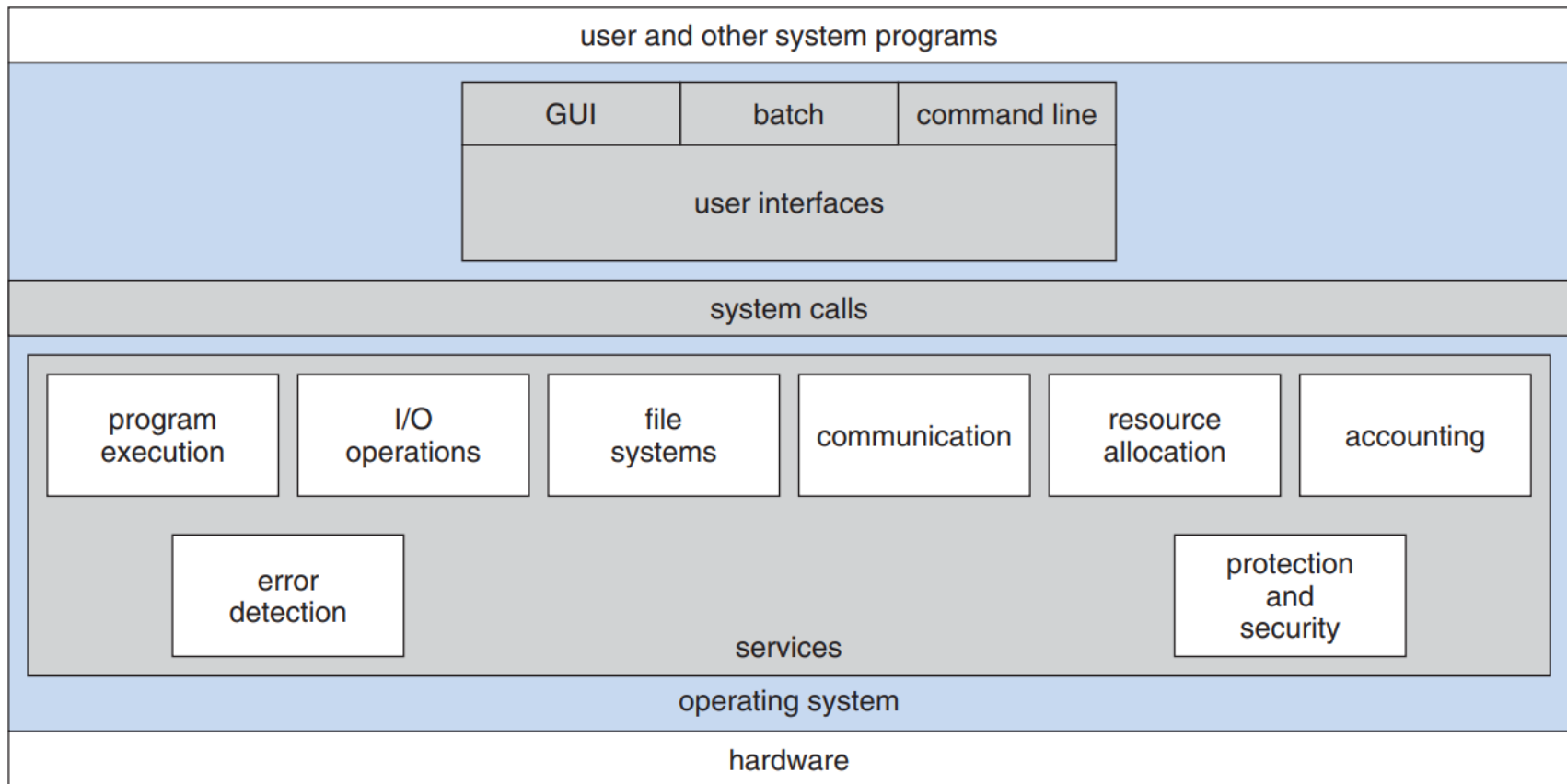
- Autenticación (Linux PAM, Windows Active Directory)
- Control de acceso (DAC, MAC - SELinux, AppArmor)
- Auditoría y logging
- Cifrado (dm-crypt, BitLocker)

Ejemplo moderno: TPM 2.0, Secure Boot, measured boot

Objetivos del Sistema Operativo

- Objetivos del sistema operativo:
 - Facilitar el uso al usuario (interfaz)
 - Facilitar la programación al programador (máquina virtual extendida)
 - Ejecutar programas eficientemente (gestor de recursos)
 - Conveniencia
 - Proveer un medio ambiente al usuario para ejecutar sus programas, por ejemplo: en computadoras personales.
 - Eficiencia
 - Incrementar la productividad de los recursos de producción.

Servicios del SO y cómo se relacionan



Silberschatz, A., Galvin, P. B., Gagne, G. (2014). Operating System Concepts. Wiley.

Ejercicio

- Cierta computadora, dispone de un procesador, cada cierto tiempo llega un proceso y se pueden ejecutar según los tiempos presentados en la siguiente tabla.

Proceso	Tiempo Llegada	Tiempo Servicio
1	0	3
2	2	2
3	4	4
4	7	3
5	12	5

1. Calcule el tiempo de término de los procesos, asumiendo que **se ejecutan en secuencia**.
2. Calcule el tiempo de término, asumiendo que dos procesos **se pueden ejecutar en paralelo**.

Ejercicio

1

Proceso	Tiempo Llegada	Tiempo Servicio	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1	0	3																	
2	2	2																	
3	4	4																	
4	7	3																	
5	12	5																	

Proceso	Tiempo Llegada	Tiempo Servicio	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1	0	3																	
2	2	2			→														
3	4	4				→													
4	7	3								→									
5	12	5																	

2

Proceso	Tiempo Llegada	Tiempo Servicio	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1	0	3																	
2	2	2			→														
3	4	4				→													
4	7	3								→									
5	12	5																	

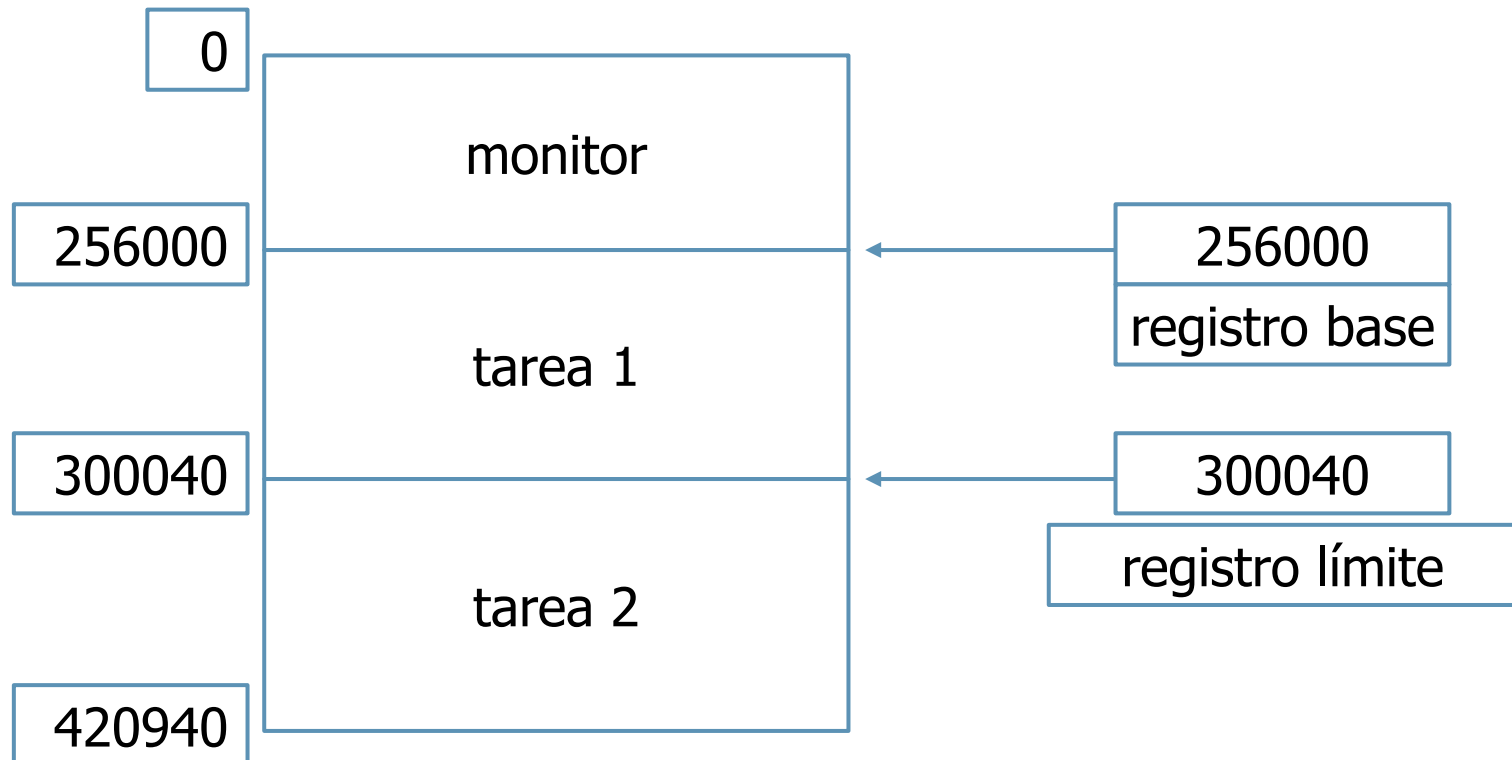
Modo dual de operación

- Al compartir recursos
 - El SO asegura que un programa no afectará a otro
 - ¿Cómo?
- Solución:
 - Mecanismo de protección hardware (bit) que permite, al menos, 2 modos de ejecución: usuario (1) y sistema (0).
- Instrucciones privilegiadas sólo posibles en modo sistema
- Cuando hay interrupción, trap o excepción, el hardware cambia a modo sistema

Protección por hardware

- E/S:
 - Siempre instrucciones privilegiadas, modo sistema
- Memoria:
 - Registros base y límite por programa
 - Accesos fuera originan un fallo de protección
- CPU:
 - Temporizador que interrumpe periódicamente para dar control al SO y que decida

Protección por hardware



Arquitectura general del sistema

Si las instrucciones de E/S son privilegiadas, ¿cómo ejecuta el usuario la E/S?

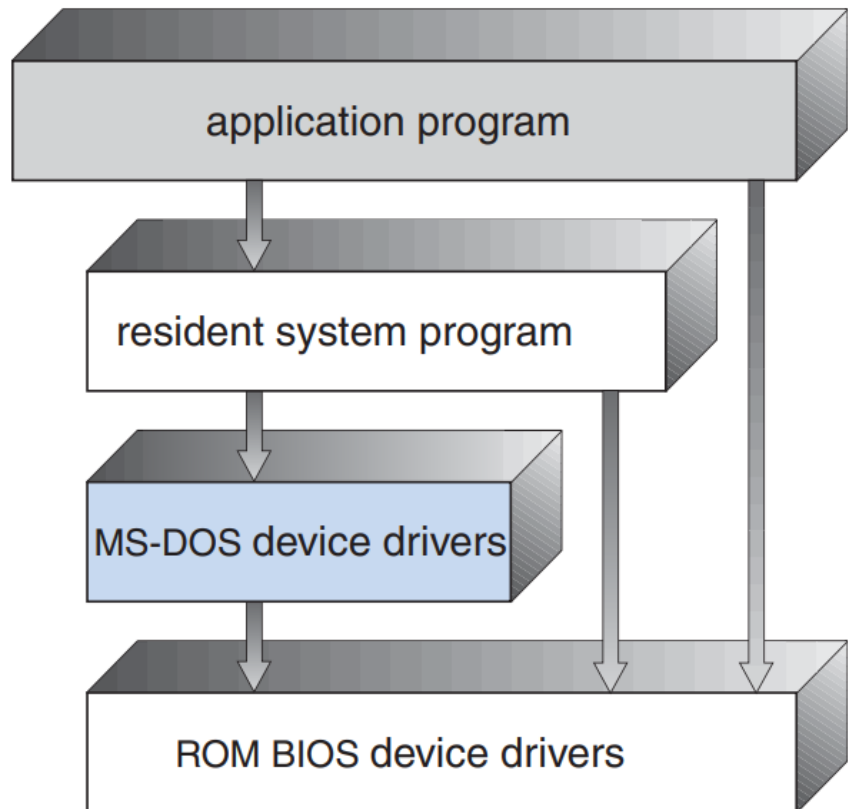
- Llamada al sistema
 - Método estándar de acceso a los servicios del SO
 - El programa genera un trap (interrupción sw)
 - Se salta a ejecutar la RTI (Rutina de Tratamiento de Interrupción, un servicio del SO) y a la vez se cambia a modo sistema
 - El SO verifica los parámetros, ejecuta la petición y devuelve control en la instrucción siguiente
- El SO oculta las llamadas al sistema en bibliotecas que se instalan con el SO



Arquitecturas de Sistemas Operativos

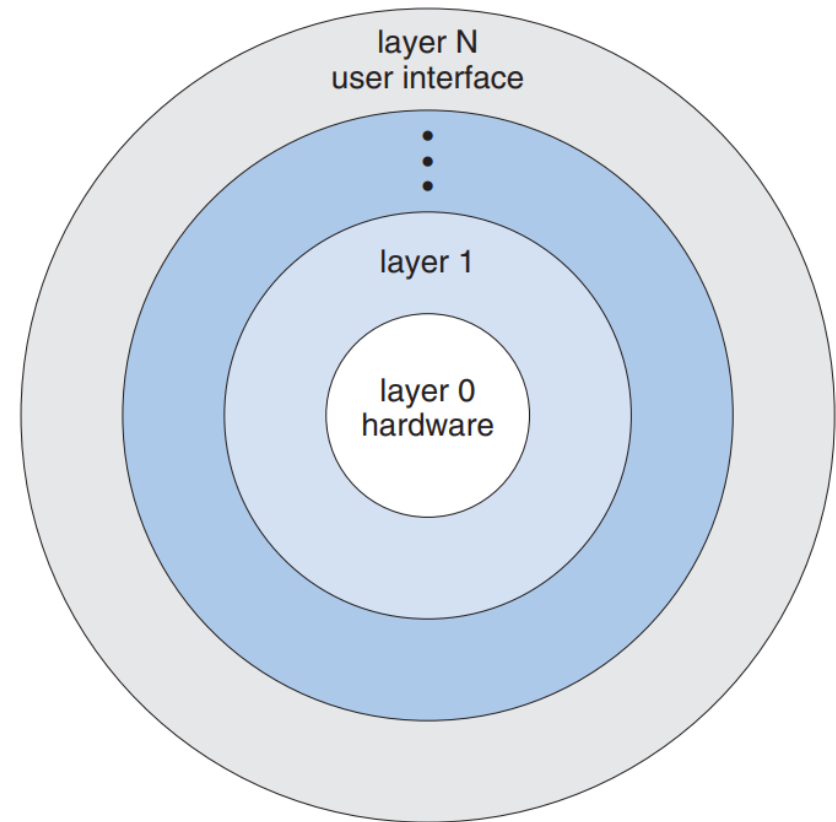
Estructura Simple

- Monolíticos
 - Todo entre las llamadas al sistema y el hardware
 - No está dividido en módulos
 - Interfaces y niveles de funcionalidad no separados
 - Ejemplo: MS-DOS
 - Escritos para funcionalidad específica y gestionar hardware reducido
 - Monoproceso

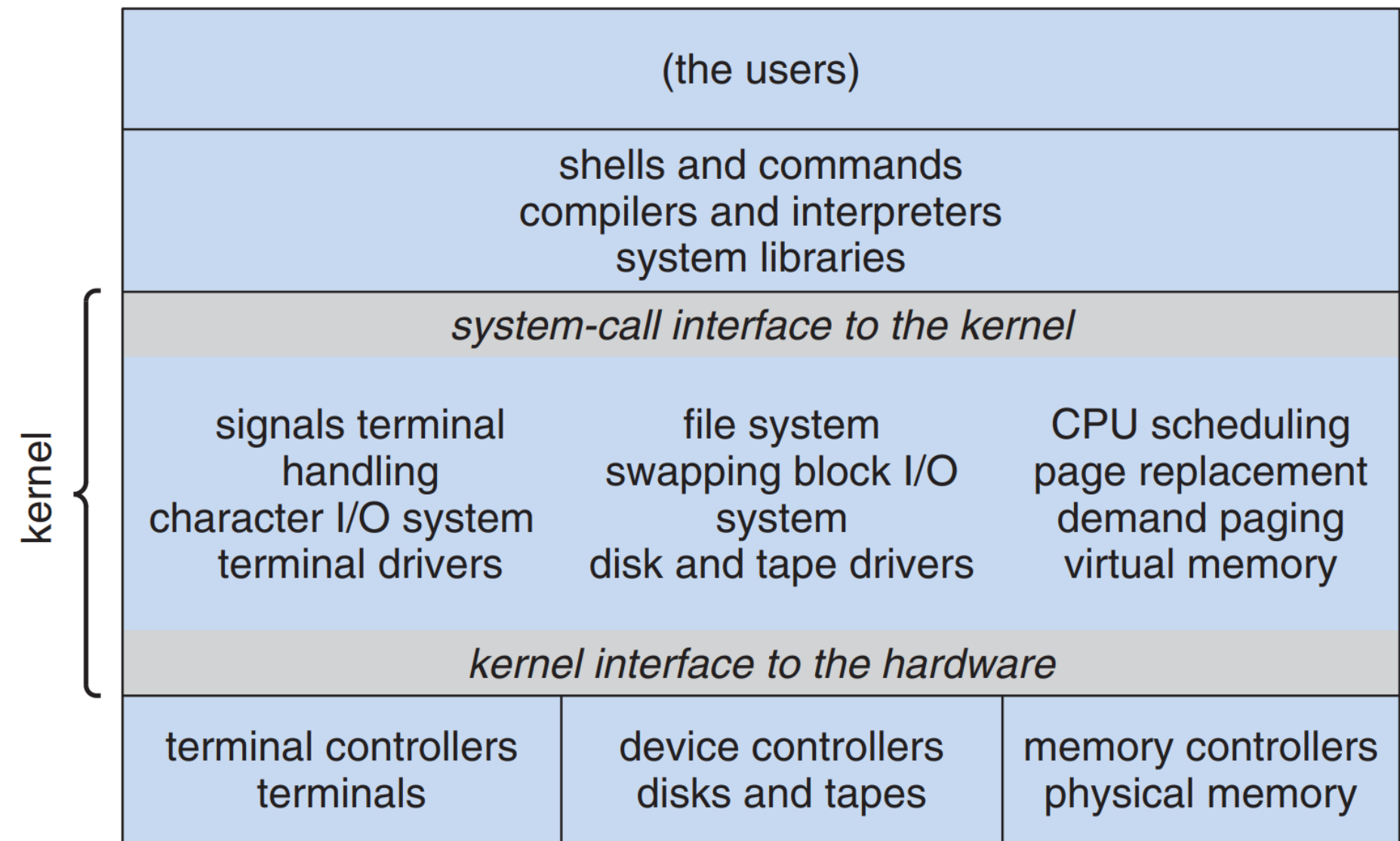


Enfoque por capas

- Dividido en múltiples niveles («capas de una cebolla»)
- El nivel más alto: la interfaz de llamadas
- Cada nivel usa únicamente servicios del inferior y da servicio al superior
- Ejemplo: Multics (Multiplexed Information and Computing Service, 1964)
- Potenciales problemas con el orden de las llamadas entre capas



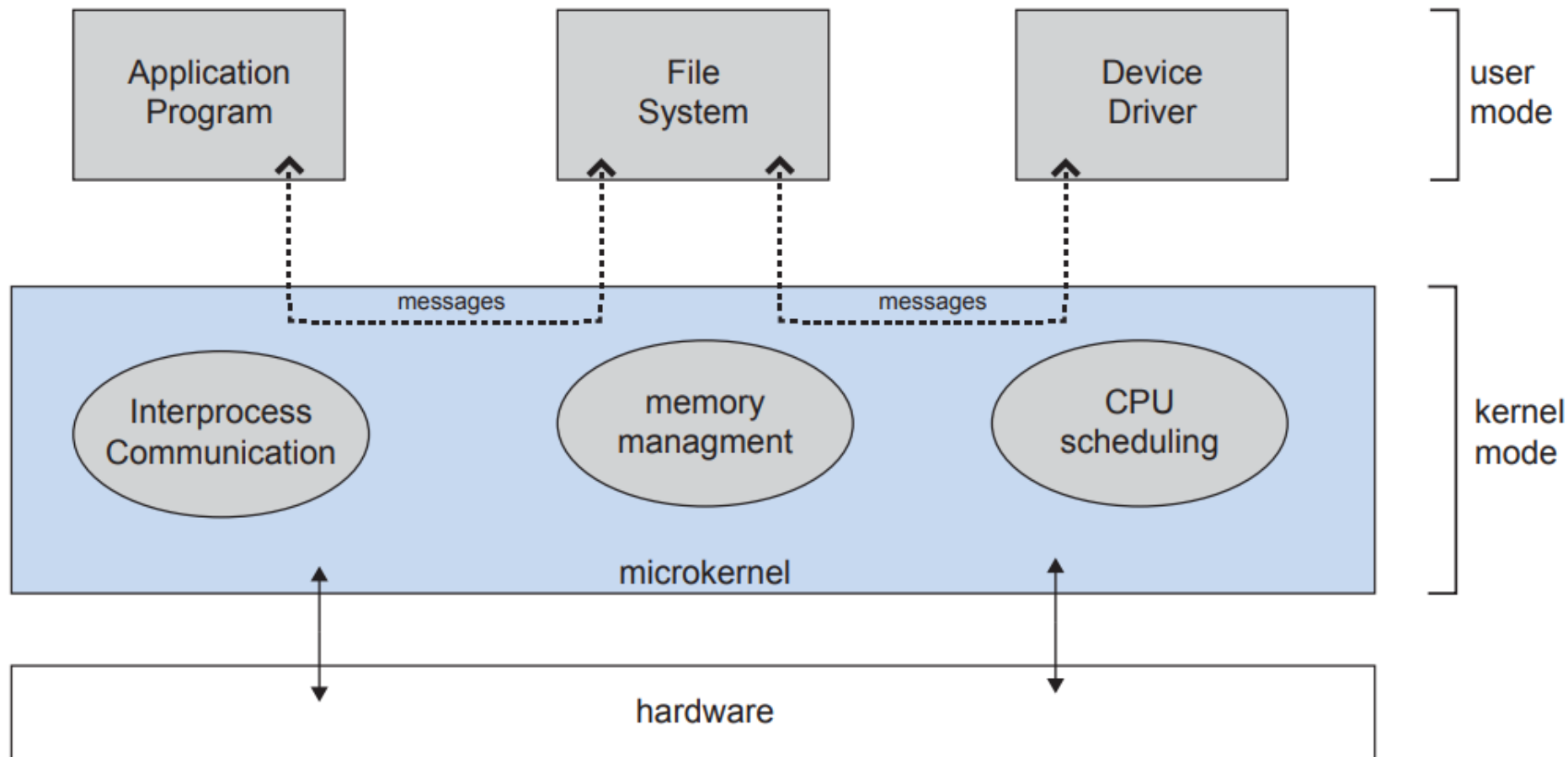
Estructura Simple



Silberschatz, A., Galvin, P. B., Gagne, G. (2014). Operating System Concepts. Wiley.

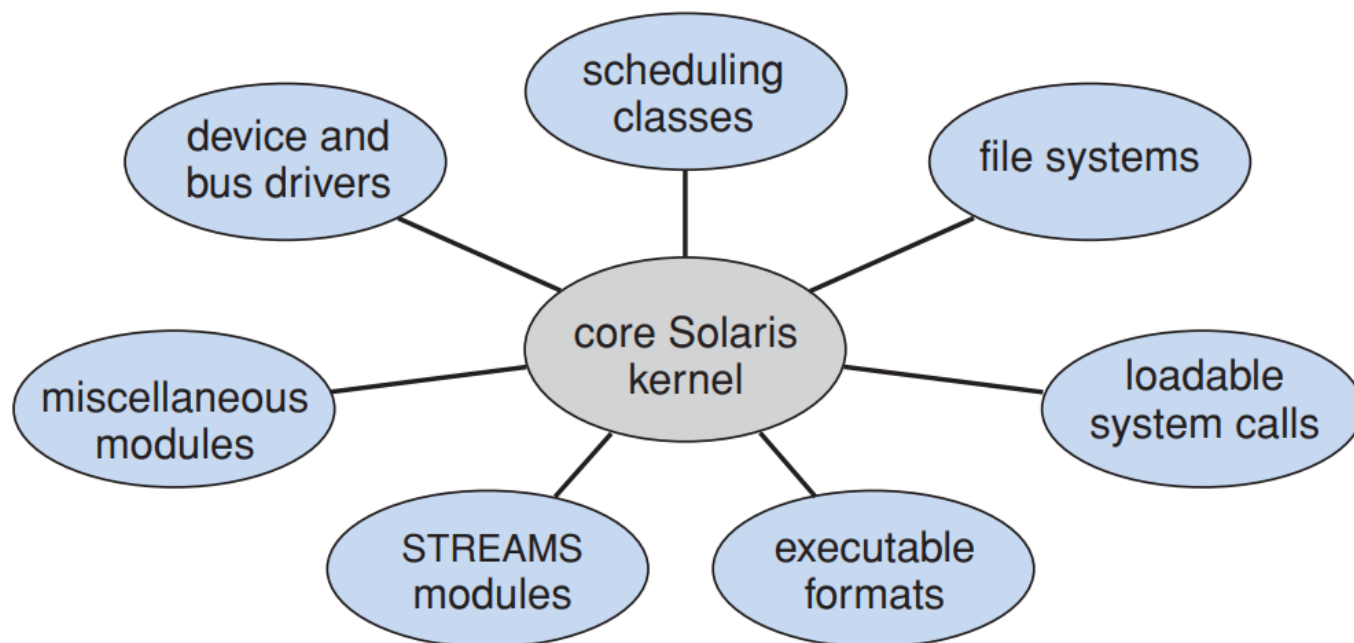
Sistemas con micronúcleo (microkernel)

- Reducido a las funcionalidades mínimas imprescindibles
- Se sacan fuera del núcleo los demás servicios
- Comunicación vía paso de mensajes



Sistemas modulares

- Dividido en módulos con funcionalidad e interfaces claramente definidos
- Modular y estructurado
- Se pueden sustituir componentes por otros con el mismo interfaz
- Muchas funcionalidades
- Multiproceso y multiusuario



Próxima lección

- Introducción Sistemas Operativos (continuación)
 - Estructura del Sistema Operativo
 - Maquinas virtuales
- Administración de Procesos
 - Conceptos
 - Comunicación entre procesos
 - Threads
 - Planificación del CPU
 - Sincronización de Procesos
 - Bloqueos

MUCHAS GRACIAS